

Assignment_03

December 4, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[54]: import math
      # Imports
      import numpy as np
      import matplotlib.pyplot as plt
      import networkx as nx
      import os
      import glob
      import re
      import random
      import multiprocessing
      from statistics import mean

      import scipy.special
      from tqdm import tqdm
      import shutil
      from joblib import Parallel, delayed
      from scipy.special import factorial
      import powerlaw
      from NEMtropy import DirectedGraph, UndirectedGraph
      from NEMtropy import matrix_generator as mg
      from NEMtropy.network_functions import build_adjacency_from_edgelist
```

```
[55]: # Define colors with reduced opacity from a colormap
      COLOR_BLUE = 'dodgerblue'
      COLOR_GREEN = 'palegreen'
      COLOR_RED = 'lightcoral'
      COLOR_YELLOW = 'gold'
      COLOR_GREY = 'slategrey'
      COLOR_BLACK = 'black'
      COLOR_LIGHT_GRAY = 'lightgray'
      COLOR_DARK_GRAY = 'darkgray'

      # Define the main data directory
```

```
OUTPUT_PATH = "./output"
```

```
[56]: # Set Seed  
SEED = 42
```

```
[57]: # Prepare output folder  
  
# Delete the entire 'output' directory and its contents  
if os.path.exists(OUTPUT_PATH):  
    shutil.rmtree(OUTPUT_PATH)  
  
# Create the 'output' directory  
os.makedirs(OUTPUT_PATH)  
  
# Define subdirectories  
OUTPUT_PATH_ERGMS = os.path.join(OUTPUT_PATH, "ERGMS")  
  
# Recreate the subdirectories  
os.makedirs(OUTPUT_PATH_ERGMS)
```

```
[58]: # Define the directory containing the .gml files  
directory = 'assignment_03_data'  
  
# Use glob to get a list of all .gml files in the directory  
gml_files = glob.glob(os.path.join(directory, '*.gml'))  
  
# Init an empty dictionary for all graphs - the key is the actual name, and the  
# value is the graph data  
graphs = {}  
  
# Use tqdm to show a progress bar in Jupyter Notebook  
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:  
    # Loop through the list of .gml files and load them  
    for file_path in gml_files:  
        with open(file_path, 'r') as file:  
            content = file.read()  
  
            # Use regex to extract the desired part  
            match = re.search(r'/(~/+)\.gml$', file_path)  
            if match:  
                graph_name = match.group(1)  
  
                # Add to the graph dictionary  
                graph = nx.read_gml(file_path)  
                graphs[graph_name] = graph
```

```

        # Print the graph name when each part is finished
        print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML Files: 50%| | 1/2 [00:00<00:00, 4.52it/s]

Finished loading graph_eu_airlines

Loading GML Files: 100%| | 2/2 [00:00<00:00, 2.14it/s]

Finished loading graph_power

```

[59]: # Define the directory containing the .gml files
directory = 'assignment_03_data/World_Trade_Web'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.graphml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
# value is the graph data
graphs_wtn = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML WTW Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/+)\.txt.graphml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_graphml(file_path)
                graphs_wtn[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML WTW Files: 9%| | 1/11 [00:00<00:03, 2.52it/s]

Finished loading WDN_1997

Loading GML WTW Files: 18%| | 2/11 [00:06<00:35, 3.92s/it]

```

Finished loading WDN_1992
Finished loading WDN_2000

Loading GML WTW Files: 45%|          | 5/11 [00:08<00:07, 1.28s/it]

Finished loading WDN_1998
Finished loading WDN_1994

Loading GML WTW Files: 55%|          | 6/11 [00:08<00:04, 1.14it/s]

Finished loading WDN_1993
Finished loading WDN_2002

Loading GML WTW Files: 73%|          | 8/11 [00:10<00:02, 1.16it/s]

Finished loading WDN_1996

Loading GML WTW Files: 82%|          | 9/11 [00:10<00:01, 1.42it/s]

Finished loading WDN_1995

Loading GML WTW Files: 91%|          | 10/11 [00:11<00:00, 1.14it/s]

Finished loading WDN_1999
Finished loading WDN_2001

Loading GML WTW Files: 100%|         | 11/11 [00:12<00:00, 1.09s/it]

```

2 I ERGMs

```
[60]: graphs
```

```
[60]: {'graph_eu_airlines': <networkx.classes.graph.Graph at 0x1a541fc90>,
      'graph_power': <networkx.classes.graph.Graph at 0x1a8817f90>}
```

```
[61]: graphs_wtn
```

```
[61]: {'WDN_1997': <networkx.classes.digraph.DiGraph at 0x1a479ee90>,
      'WDN_1992': <networkx.classes.digraph.DiGraph at 0x1a379f2d0>,
      'WDN_2000': <networkx.classes.digraph.DiGraph at 0x1a8f4fad0>,
      'WDN_1998': <networkx.classes.digraph.DiGraph at 0x1a4d3cbd0>,
      'WDN_1994': <networkx.classes.digraph.DiGraph at 0x1aa0322d0>,
      'WDN_1993': <networkx.classes.digraph.DiGraph at 0x1a4ad5690>,
      'WDN_2002': <networkx.classes.digraph.DiGraph at 0x1ad52a190>,
      'WDN_1996': <networkx.classes.digraph.DiGraph at 0x1ab3aeb50>,
      'WDN_1995': <networkx.classes.digraph.DiGraph at 0x1ad4eaa10>,
      'WDN_1999': <networkx.classes.digraph.DiGraph at 0x1b0733310>,
      'WDN_2001': <networkx.classes.digraph.DiGraph at 0x1b0fb5bd0>}
```

2.1 1. Compute the strength assortativity coefficient on the data.

Hint: Pay attention: the underlying data (WTW) are weighted and directed, meaning you should account for four different kind of assortativity: in-in, in-out, out-in, out-out.

```
[62]: def calculate_assortativity(graph):
    in_in = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='in', y='in', weight='weight')
    in_out = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='in', y='out', weight='weight')
    out_in = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='out', y='in', weight='weight')
    out_out = nx.assortativity.degree_pearson_correlation_coefficient(G=graph,
↪x='out', y='out', weight='weight')

    # Create a dictionary to store assortativity values
    assortativity_dict = {
        "In-In Assortativity": in_in,
        "In-Out Assortativity": in_out,
        "Out-In Assortativity": out_in,
        "Out-Out Assortativity": out_out
    }

    # Return dictionary with assortativity values
    return assortativity_dict
```

```
[ ]: assortativities = {}
for graph_name, graph in graphs_wtn.items():
    assortativities[graph_name] = calculate_assortativity(graph=graph)

print(assortativities)
```

```
[64]: # Extracting graph names and assortativity values
graph_names = list(assortativities.keys())
keys = list(assortativities[graph_names[0]].keys()) # Extracting keys for
↪assortativity values
values = {graph_name: [assortativities[graph_name][key] for key in keys] for
↪graph_name in graph_names}

# Creating side-by-side bar plots for each network
bar_width = 0.2 # Width of each bar
index = np.arange(len(graph_names)) # Index for the networks

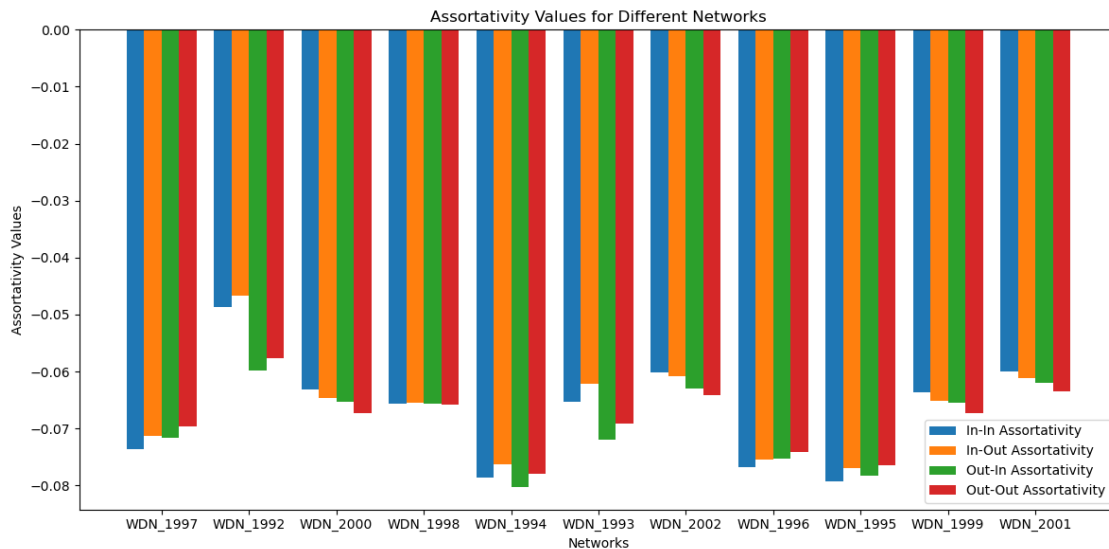
plt.figure(figsize=(12, 6))

for i, key in enumerate(keys):
    plt.bar(index + (i * bar_width), [values[graph_name][i] for graph_name in
↪graph_names], bar_width, label=key)

plt.xlabel('Networks')
plt.ylabel('Assortativity Values')
plt.title('Assortativity Values for Different Networks')
```

```
plt.xticks(index + (bar_width * (len(keys) - 1) / 2), graph_names)
plt.legend()
plt.tight_layout()

file_name=f"assortativity-values-for-internet-networks.png"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()
```



2.2 2. Fit the Undirected Enhanced CM and Directed Enhanced CM using the CReMa method.

Hint: You can use the `nentropy` package on `networkx` to deploy these methods. You can find some examples at <https://github.com/nicoloval/NEMtropy/blob/master/examples/Directed%20Graphs>. `ipynb`. For additional questions, do not hesitate to ask at the QA.

```
[65]: # Just for understanding (from exercise)

# DIRECTED: (G: nx graph; graph_U NEMtropy graph)
# adj = nx.to_numpy_array(G)
# graph_D = DirectedGraph(adj)

# UNDIRECTED: via flipping adj matrix (G: nx graph; graph_U NEMtropy graph)
# adj = nx.to_numpy_array(G)
# adj_sym = adj + np.transpose(adj)
# graph_U = UndirectedGraph(adj_sym)
```

```
# UNDIRECTED: via to_undirected() (G: nx graph; graph_U NEMtropy graph)
# G = G.to_undirected()
# adj = nx.to_numpy_array(G)
# graph_U = UndirectedGraph(adj)
```

```
[66]: # Create undirected versions of the graph

graphs_wtn_undirected = {}
for graph_name, graph in graphs_wtn.items():
    # We can use this or either symmetrize the adj matrix
    # Since we use that, we don't need to adj_sym = adj + np.transpose(adj)
    ↪when extracting the adj from an undirected graph
    graph_undirected = graph.to_undirected()

    graphs_wtn_undirected[graph_name] = graph_undirected
```

```
[67]: FIG_SIZE = (16, 12)
      NODE_SIZE = 100
      MAX_STEPS = 10000
```

```
[ ]: graphs_wtn
```

```
[ ]: graphs_wtn_undirected
```

```
[70]: nemtropy_directed_results = {}
      nemtropy_undirected_results = {}
```

```
[71]: def to_entropy_graph(graph):
      # Extract necessary information from the NetworkX graph
      adj_weigh = nx.to_numpy_array(graph) # Adjacency matrix
      edges = list(graph.edges()) # Edge list
      degrees = dict(graph.degree()) # Degree sequence (as a dictionary)
      strengths = dict(graph.degree(weight='weight')) # Strength sequence
      ↪(assuming weights are present)

      return adj_weigh, edges, degrees, strengths
```

```
[72]: def to_networkx_graph(adj_weigh):
      graph = nx.from_numpy_array(adj_weigh, create_using=nx.DiGraph)

      # Not needed anymore
      # Set edge attributes (weights) from the adjacency matrix
      edges = graph.edges(data=True)
      for (u, v, d) in edges:
          d['weight'] = adj_weigh[u][v]
      return graph
```

```
[73]: # Helper
def plot_networkx(graph, graph_name, is_directed):

    # Plot the graph
    plt.figure(figsize=FIG_SIZE)

    # Set the layout for consistent plotting
    pos = nx.kamada_kawai_layout(graph)

    # Draw nodes and labels
    nx.draw(graph, pos, with_labels=True, node_size=NODE_SIZE,
    ↪node_color='skyblue', font_weight='bold')

    # Draw edge labels with weights
    edge_labels = nx.get_edge_attributes(graph, 'weight')
    nx.draw_networkx_edge_labels(graph, pos, edge_labels=edge_labels)

    plt.title(f'{ "Undirected" if not is_directed else "Directed" } Weighted_
    ↪Graph of Network {graph_name} (networkx plot)')
    file_name=f'{ "undirected_" if not is_directed else
    ↪"directed_" }weighted_graph_{graph_name}_networkx.png'
    complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
    plt.savefig(complete_plot_filename)
    plt.show()
```

```
[74]: # Helper
def plot_entropy_graph(adj_weigh, graph_name, is_directed):
    # Create a NetworkX graph from the weighted adjacency matrix
    graph = nx.from_numpy_array(adj_weigh, create_using=nx.DiGraph)

    # Set edge attributes (weights) from the adjacency matrix
    edges = graph.edges(data=True)
    for (u, v, d) in edges:
        d['weight'] = adj_weigh[u][v]

    # Plot the graph
    plt.figure(figsize=FIG_SIZE)

    pos = nx.spring_layout(graph, seed=SEED) # Set the layout for consistent_
    ↪plotting

    # Draw nodes and labels
    nx.draw(graph, pos, with_labels=True, node_size=NODE_SIZE,
    ↪node_color='skyblue', font_weight='bold')

    # Draw edge labels with weights
    edge_labels = nx.get_edge_attributes(graph, 'weight')
```

```

nx.draw_networkx_edge_labels(graph, pos, edge_labels=edge_labels)

plt.title(f'{"Undirected" if not is_directed else "Directed" } Weighted_
↳Graph of Network {graph_name} (networkx plot)')
file_name=f'{"undirected_" if not is_directed else_
↳"directed_"}weighted_graph_{graph_name}_networkx.png'
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```

```

[75]: # Helper
def pretty_print_nentropy_results(results_dict):
    for graph_name, results in results_dict.items():
        nentropy_graph = results['nentropy_graph']
        fit = results['fit']
        error_degree = results['error_degree']
        error_strength = results['error_strength']
        rel_error_degree = results['relative_error_degree']
        rel_error_strength = results['relative_error_strength']

        print(f"Graph Name: {graph_name}")
        print(f"Error Degree: {error_degree}")
        print(f"Error Strength: {error_strength}")
        print(f"Relative Error Degree: {rel_error_degree}")
        print(f"Relative Error Strength: {rel_error_strength}")
        print("-----")

```

```

[ ]: for graph_name, graph in tqdm(graphs_wtn.items(), total=len(graphs_wtn.
↳items()), desc='Processing directed graphs'):

    # Plot the networkx graph
    # plot_networkx(graph=graph, graph_name=graph_name, is_directed=True)

    # Extract necessary information from the NetworkX graph
    adj_weigh, edges, degrees, strengths = to_entropy_graph(graph)

    # Instantiate DirectedGraph with the extracted information
    nentropy_graph = DirectedGraph(
        adjacency=adj_weigh,
        # Leave this out, since TA did the same
        #edgelist=edges,
        #degree_sequence=np.array(list(degrees.values())),
        #strength_sequence=np.array(list(strengths.values()))
    )

    # Plot the entropy graph

```

```

    # plot_entropy_graph(adj_weigh=adj_weigh, graph_name=graph_name,
    ↪is_directed=True)

    # We can the see the graph sequence of the graph by tapping
    #print("Out strength sequence ",nemtropy_graph.out_strength)
    #print("In strength sequence ",nemtropy_graph.in_strength)

    # Create the fit
    fit = nemtropy_graph.solve_tool(model="crema", # TODO: or dcn_exp?
        method="newton",
        initial_guess="random",
        adjacency="dcm_exp",
        method_adjacency="newton",
        max_steps=MAX_STEPS) # maybe overkill

    nemtropy_directed_results[graph_name] = {'nemtropy_graph': nemtropy_graph
    ↪, 'fit': fit, 'error_degree': nemtropy_graph.error_degree, 'error_strength':
    ↪nemtropy_graph.error_strength, 'relative_error_degree': nemtropy_graph.
    ↪relative_error_degree, 'relative_error_strength': nemtropy_graph.
    ↪relative_error_strength, }

```

```
[77]: pretty_print_nemtropy_results(nemtropy_directed_results)
```

```

Graph Name: WDN_1997
Error Degree: 3.6638425626733806e-09
Error Strength: 107564.59875488281
Relative Error Degree: 1.0703497347700856e-10
Relative Error Strength: 9.583487359438782e-06
-----
Graph Name: WDN_1992
Error Degree: 2.3851853825362923e-09
Error Strength: 343561.71673583984
Relative Error Degree: 3.910139971370971e-11
Relative Error Strength: 1.488109017333447e-06
-----
Graph Name: WDN_2000
Error Degree: 3.1629561192403344e-09
Error Strength: 303327.7660522461
Relative Error Degree: 1.317898383016806e-10
Relative Error Strength: 0.0012720940196702085
-----
Graph Name: WDN_1998
Error Degree: 2.4755024696787586e-09
Error Strength: 239821.06768798828
Relative Error Degree: 5.001115571051034e-11
Relative Error Strength: 0.0001333290305907048
-----
Graph Name: WDN_1994

```

Error Degree: 4.13637479823592e-09
Error Strength: 118861.97290039062
Relative Error Degree: 1.3787915994119733e-10
Relative Error Strength: 1.966097260638403e-06

Graph Name: WDN_1993
Error Degree: 3.208192822512501e-09
Error Strength: 122338.96789550781
Relative Error Degree: 1.188219563893519e-10
Relative Error Strength: 6.817327025395706e-05

Graph Name: WDN_2002
Error Degree: 3.6492728838766197e-09
Error Strength: 154190.61853027344
Relative Error Degree: 1.9206699388824314e-10
Relative Error Strength: 3.3974284626500237e-06

Graph Name: WDN_1996
Error Degree: 2.4397301956469164e-09
Error Strength: 118942.51055908203
Relative Error Degree: 8.132433985489721e-11
Relative Error Strength: 0.00011153452651484621

Graph Name: WDN_1995
Error Degree: 3.3731453186192084e-09
Error Strength: 341901.0478515625
Relative Error Degree: 8.059609018820755e-11
Relative Error Strength: 2.6246733303865888e-05

Graph Name: WDN_1999
Error Degree: 3.1799061162018916e-09
Error Strength: 125293.87316894531
Relative Error Degree: 3.5021615237459024e-11
Relative Error Strength: 0.0007581315300711763

Graph Name: WDN_2001
Error Degree: 5.437527761387173e-09
Error Strength: 151261.49584960938
Relative Error Degree: 2.364142504950945e-10
Relative Error Strength: 2.770415582715348e-05

```
[ ]: for graph_name, graph in tqdm(graphs_wtn_undirected.items()),  
      ↪total=len(graphs_wtn_undirected.items()), desc='Processing undirected_  
      ↪graphs'):  
  
      # Plot the networkx graph
```

```

# plot_networkx(graph=graph, graph_name=graph_name, is_directed=False)

# Extract necessary information from the NetworkX graph
adj_weigh, edges, degrees, strengths = to_entropy_graph(graph)

# Instantiate UndirectedGraph with the extracted information
nemtrophy_graph = UndirectedGraph(
    adjacency=adj_weigh,
    # Leave this out, since TA did the same
    #edgelist=edges,
    #degree_sequence=np.array(list(degrees.values())),
    #strength_sequence=np.array(list(strengths.values()))
)

# Plot the entropy graph
# plot_entropy_graph(adj_weigh=adj_weigh, graph_name=graph_name,
↪is_directed=False)
# We can see the graph sequence of the graph by tapping

# Create the fit
fit = nemtrophy_graph.solve_tool(model="cm_exp",
                                method="newton",
                                initial_guess="random",
                                adjacency="dcm_exp",
                                method_adjacency="newton",
                                max_steps=MAX_STEPS) # maybe overkill

nemtrophy_undirected_results[graph_name] = {'nemtrophy_graph': nemtrophy_graph,
↪ 'fit': fit, 'error_degree': nemtrophy_graph.error_degree, 'error_strength':
↪ nemtrophy_graph.error_strength, 'relative_error_degree': nemtrophy_graph.
↪ relative_error_degree, 'relative_error_strength': nemtrophy_graph.
↪ relative_error_strength, }

```

```
[ ]: pretty_print_nemtrophy_results(nemtrophy_undirected_results)
```

2.3 3. Sample 30 networks from the obtained ERGM distributions and measure the strength assortativity on all of them. Calculate average and standard deviation of each assortativity typology.

```
[80]: SAMPLE_SIZE = 30
```

```
[81]: def process_graph(graph_name, results, is_directed):
graph = results['nemtrophy_graph']
error_degree = results['error_degree']
error_strength = results['error_strength']

```

```

# Generate 30 ensembles copies using ensemble_sampler
output_dir = f"{OUTPUT_PATH_ERGMS}/{graph_name}-{'directed' if is_directed_
↪else 'undirected'}-sample/"
graph.ensemble_sampler(SAMPLE_SIZE, cpu_n=2, output_dir=output_dir)

sampled_graphs = []
assortativity_values = []
for i in range(0, SAMPLE_SIZE, 1):
    # We read the edgelist
    edgelist_ens = np.loadtxt(f"{output_dir}/{i}.txt")

    # and build the adjacency matrix
    ens_adj = build_adjacency_from_edgelist(edgelist=edgelist_ens,
                                             is_directed=is_directed,
                                             is_sparse=False,
                                             is_weighted=True)

    sampled_graph = to_networkx_graph(ens_adj)
    sampled_graphs.append(sampled_graph)

    # measure strength assortativity
    assortativities_sampled = calculate_assortativity(sampled_graph)
    assortativity_values.append(assortativities_sampled)

categories = list(assortativity_values[0].keys())

# Initialize dictionaries to store mean and standard deviation for each_
↪category
mean_values = {}
std_values = {}

# Loop through each category
for category in categories:
    # Extract values for the current category across all dictionaries
    category_values = [data[category] for data in assortativity_values]

    # Calculate mean and standard
    mean = np.mean(category_values)
    std = np.std(category_values)

    # Store mean and standard deviation for the category
    mean_values[category] = mean
    std_values[category] = std

results['samples'] = sampled_graphs
results['assortativity_means'] = mean_values
results['assortativity_stds'] = std_values

```

```

results['assortativity_data'] = assortativities[graph_name]

# Return the modified results dictionary
return results

```

```

[82]: def process_results_parallel(results_dict, is_directed):
        updated_results = Parallel(n_jobs=-1)(
            delayed(process_graph)(graph_name, results, is_directed)
            for graph_name, results in tqdm(results_dict.items(), desc=f'Processing_
↳{"directed" if is_directed else "undirected"} results')
        )

        # Update the original results_dict with modified values
        # Very nasty bug...
        for i, (graph_name, _) in enumerate(results_dict.items()):
            results_dict[graph_name] = updated_results[i]

        return results_dict

```

```

[83]: nemtropy_directed_results = process_results_parallel(nemtropy_directed_results,
↳is_directed=True)

```

```
Processing directed results: 100%|          | 11/11 [00:00<00:00, 23.07it/s]
```

```
[ ]: nemtropy_directed_results
```

```

[85]: nemtropy_undirected_results =
↳process_results_parallel(nemtropy_undirected_results, is_directed=False)

```

```
Processing undirected results: 100%|          | 11/11 [00:00<00:00, 456.68it/s]
```

```
[ ]: nemtropy_undirected_results
```

2.4 4. Plot strength assortativity as a function of time (the WTW network year), comparing the observed real value with the average and error bars obtained from the samples samples.

Hint: plot error bars as confidence intervals. Plot all pairs of assortativity for the undirected and directed case(in-in, in-out, out-out)

```
[ ]: assortativities
```

```

[117]: # Assortativity pairs
        assortativity_pairs = ['In-In Assortativity', 'In-Out Assortativity', 'Out-In_
↳Assortativity', 'Out-Out Assortativity']

```

```
[ ]: nemtropy_directed_results
```

```

[119]: # Years
years = list(nentropy_directed_results.keys())
x = np.arange(len(years))

# Plotting
plt.figure(figsize=(18, 6))

for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means = [nentropy_directed_results[year]['assortativity_means'][pair] for year in years]
    sample_std = [nentropy_directed_results[year]['assortativity_stds'][pair] for year in years]

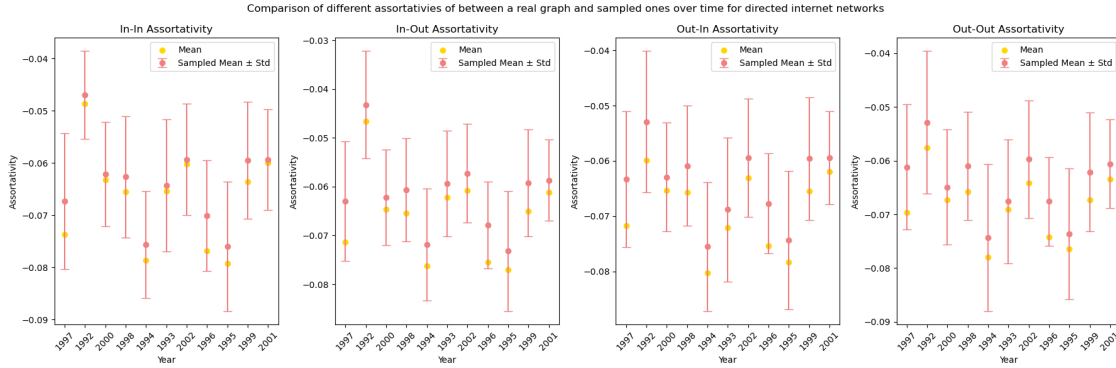
    plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Sampled Mean ± Std', color=COLOR_RED, capsize=5)

    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair] for year in years]
    plt.scatter(x, real_mean, color=COLOR_YELLOW, label='Mean')

    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')
    plt.xlabel('Year')
    plt.ylabel('Assortativity')
    plt.legend()

overall_title = "Comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name=f"assortativity-comparison-data-vs-sampled-directed-internet-networks"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```
[120]: # Years
years = list(nentropy_undirected_results.keys())
x = np.arange(len(years))

# Plotting
plt.figure(figsize=(18, 6))

for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means = [
        nentropy_undirected_results[year]['assortativity_means'][pair] for year in years
    ]
    sample_std = [nentropy_undirected_results[year]['assortativity_stds'][pair] for year in years]

    plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Simulated Mean ± Std', color=COLOR_BLUE, capsizes=5)

    # TODO: Can we use the results from the original directed data?
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair] for year in years]
    plt.scatter(x, real_mean, color=COLOR_GREEN, label='Mean')

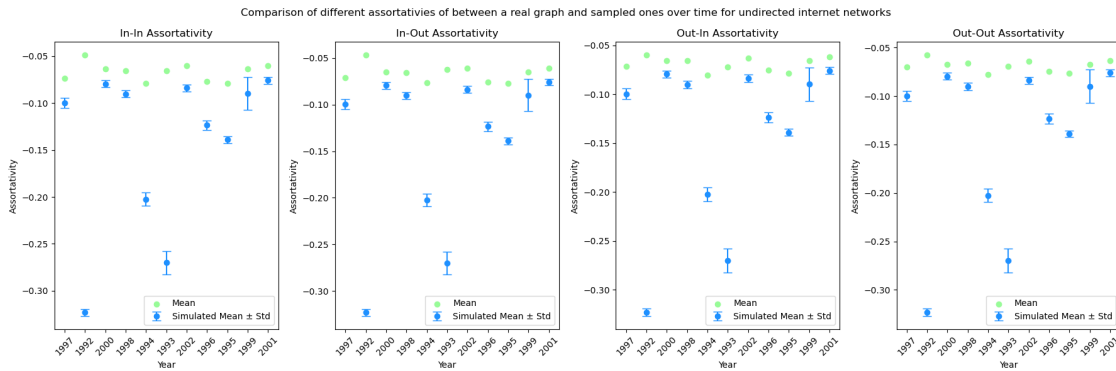
    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')
    plt.xlabel('Year')
    plt.ylabel('Assortativity')
    plt.legend()

overall_title = "Comparison of different assortativities of between a real graph and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
```

```

file_name=f"assortativity-comparison-data-vs-sampled-undirected-internet-networks"
complete_plot_filename=os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



We can see that all assortativities are the the same, which is expected in an undirected graph.

```

[121]: # Selecting a single assortativity pair
assortativity_pair = 'In-In Assortativity'

[122]: # Plotting
plt.figure(figsize=(8, 6))

sample_means =
    ↳ [nemtropy_undirected_results[year]['assortativity_means'][assortativity_pair]
    ↳ for year in years]
sample_std =
    ↳ [nemtropy_undirected_results[year]['assortativity_stds'][assortativity_pair]
    ↳ for year in years]

plt.errorbar(x, sample_means, yerr=sample_std, fmt='o', label='Simulated Mean ±
    ↳ Std', color=COLOR_BLUE, capsize=5)

# Using results from the original directed data
real_mean =
    ↳ [nemtropy_directed_results[year]['assortativity_data'][assortativity_pair]
    ↳ for year in years]
plt.scatter(x, real_mean, color=COLOR_GREEN, label='Mean')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{assortativity_pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

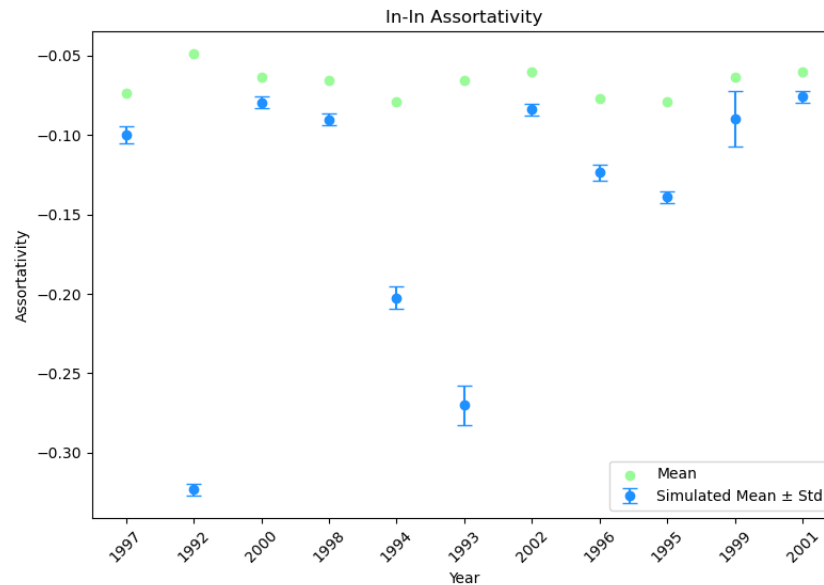
```

```

overall_title = "Comparison of different assortativities of between a real graph_
↳and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = f"assortativity-comparison-{assortativity_pair.lower().replace(' ',
↳'-')}-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```

Comparison of different assortativities of between a real graph and sampled ones over time for undirected internet networks



```

[123]: plt.figure(figsize=(18, 6))
# Linechart
for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means =
↳[nentropy_directed_results[year]['assortativity_means'][pair] for year in
↳years]
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair]
↳for year in years]

    plt.plot(x, sample_means, marker='o', label='Sampled Mean',
↳color=COLOR_RED, linestyle="--")
    plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_YELLOW,
↳linestyle="--")

```

```

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

overall_title = "Timeline comparison of different assortativities of between a_
↳real graph and sampled ones over time for directed internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = _
↳f"assortativity-timeline-comparison-real-vs-sampled-means-directed-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[124]: plt.figure(figsize=(18, 6))
# Linechart
for i, pair in enumerate(assortativity_pairs):
    plt.subplot(1, len(assortativity_pairs), i + 1)
    sample_means = _
    ↳[nentropy_undirected_results[year]['assortativity_means'][pair] for year in _
    ↳years]
    real_mean = [nentropy_undirected_results[year]['assortativity_data'][pair] _
    ↳for year in years]

    plt.plot(x, sample_means, marker='o', label='Sampled Mean', _
    ↳color=COLOR_BLUE, linestyle="--")
    plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_GREEN, _
    ↳linestyle="--")

    plt.xticks(x, [year[-4:] for year in years], rotation=45)
    plt.title(f'{pair}')

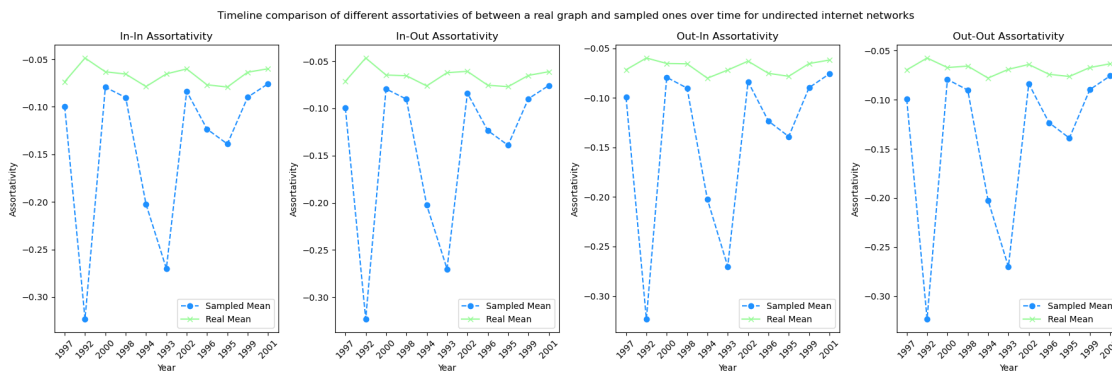
```

```

plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

overall_title = "Timeline comparison of different assortativities of between a_
↳real graph and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name =_
↳f"assortativity-timeline-comparison-real-vs-sampled-means-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[129]: plt.figure(figsize=(8, 6))

sample_means =_
↳[nemtropy_undirected_results[year]['assortativity_means'][assortativity_pair]_
↳for year in years]
real_mean =_
↳[nemtropy_undirected_results[year]['assortativity_data'][assortativity_pair]_
↳for year in years]

plt.plot(x, sample_means, marker='o', label='Sampled Mean', color=COLOR_BLUE,_
↳linestyle="--")
plt.plot(x, real_mean, marker='x', label='Real Mean', color=COLOR_GREEN,_
↳linestyle="--")

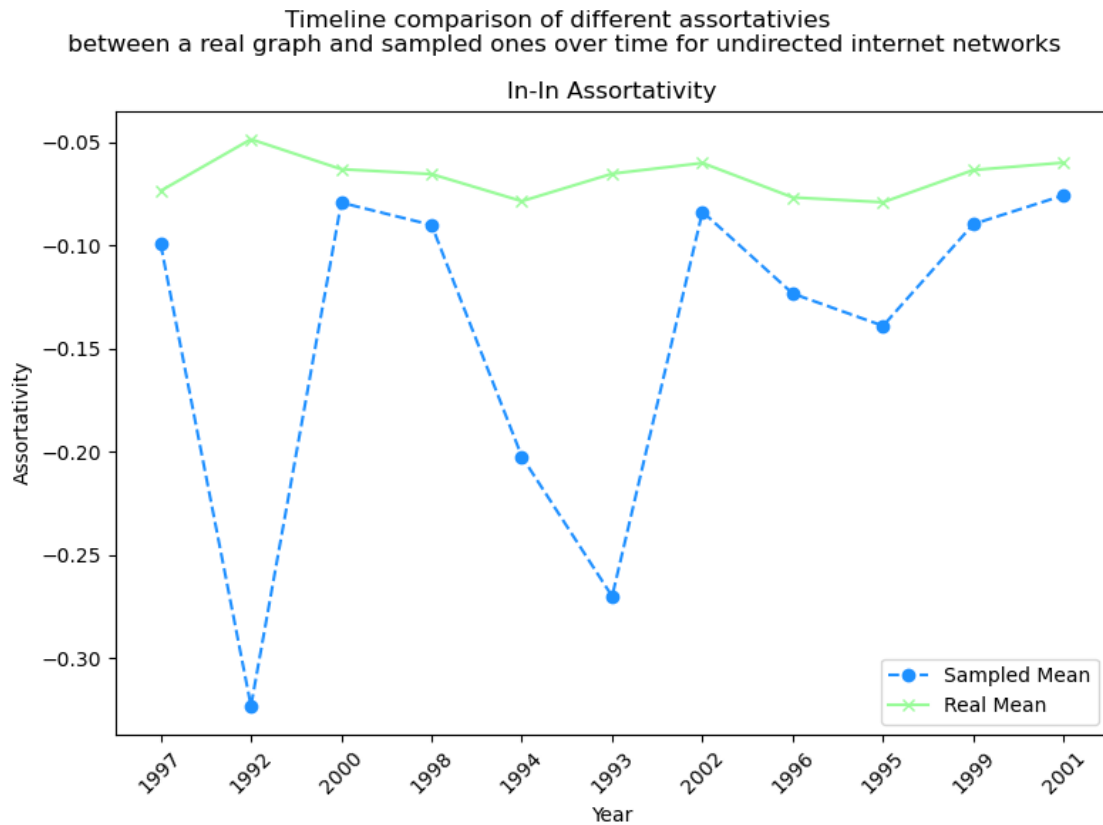
plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.title(f'{assortativity_pair}')
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.legend()

```

```

overall_title = "Timeline comparison of different assortativities \n between a_
↳real graph and sampled ones over time for undirected internet networks"
plt.suptitle(overall_title)
plt.tight_layout()
file_name = f"assortativity-timeline-comparison-{assortativity_pair.lower().
↳replace(' ', '-')}-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[126]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)))

for i, pair in enumerate(assortativity_pairs):
    sample_means =
↳[nentropy_directed_results[year]['assortativity_means'][pair] for year in
↳years]

```

```

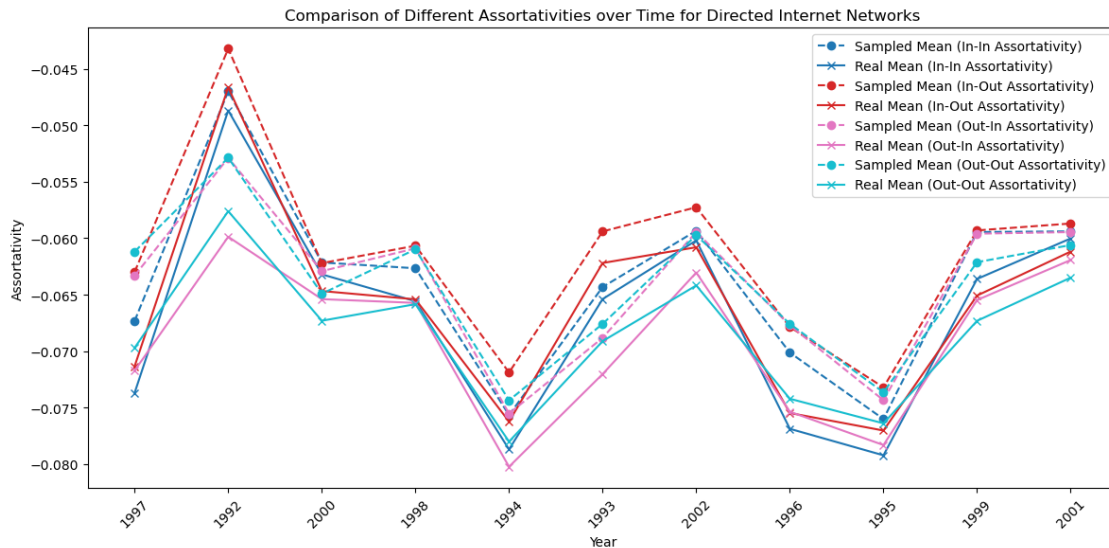
    real_mean = [nentropy_directed_results[year]['assortativity_data'][pair]
    ↪for year in years]

    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({pair})',
    ↪color=colors[i], linestyle='--')
    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
    ↪color=colors[i], linestyle='-')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Directed
    ↪Internet Networks')
plt.legend()
plt.tight_layout()

file_name =
    ↪f"assortativity-all-comparison-real-vs-sampled-means-directed-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[127]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)))

for i, pair in enumerate(assortativity_pairs):

```

```

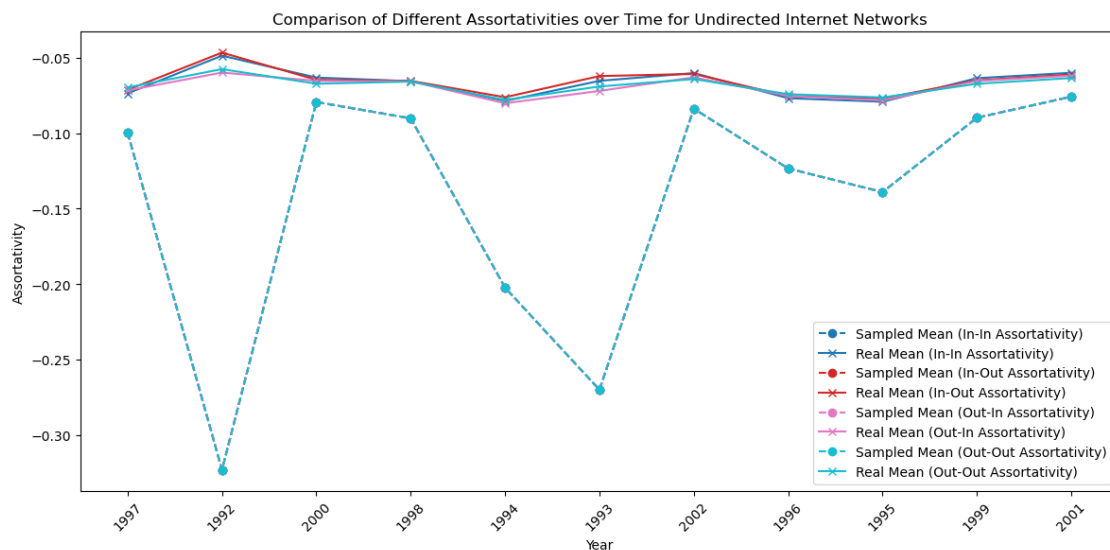
    sample_means =
    ↪[nemtropy_undirected_results[year]['assortativity_means'][pair] for year in
    ↪years]
    real_mean = [nemtropy_undirected_results[year]['assortativity_data'][pair]
    ↪for year in years]

    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({pair})',
    ↪color=colors[i], linestyle='--')
    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
    ↪color=colors[i], linestyle='-')

plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Undirected
    ↪Internet Networks')
plt.legend()
plt.tight_layout()

file_name =
    ↪f"assortativity-all-comparison-real-vs-sampled-means-undirected-internet-networks"
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



```

[128]: plt.figure(figsize=(12, 6))

colors = plt.cm.tab10(np.linspace(0, 1, len(assortativity_pairs)+1))

```

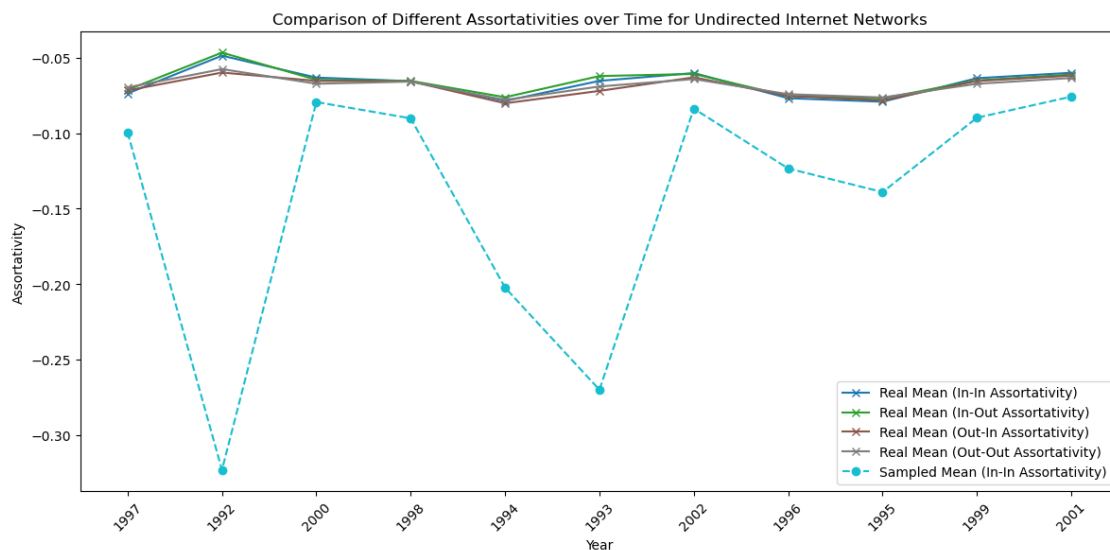
```

i = 0
for i, pair in enumerate(assortativity_pairs):
    sample_means = []
    for year in years:
        real_mean = [nemtropy_undirected_results[year]['assortativity_data'][pair]
                      for year in years]

    plt.plot(x, real_mean, marker='x', label=f'Real Mean ({pair})',
             color=colors[i], linestyle='-')
    plt.plot(x, sample_means, marker='o', label=f'Sampled Mean ({assortativity_pair})',
             color=colors[i+1], linestyle='--')
plt.xticks(x, [year[-4:] for year in years], rotation=45)
plt.xlabel('Year')
plt.ylabel('Assortativity')
plt.title('Comparison of Different Assortativities over Time for Undirected Internet Networks')
plt.legend()
plt.tight_layout()

file_name = []
for i in range(10):
    file_name.append(f"assortativity-reduced-comparison-real-vs-sampled-means-undirected-internet-networks-{i}.png")
complete_plot_filename = os.path.join(OUTPUT_PATH_ERGMS, file_name)
plt.savefig(complete_plot_filename)
plt.show()

```



2.5 5. Write a short paragraph to comment on the interpretation of strength assortativity for this dataset, and about the conclusions you can draw via the inference you made using the ERGM models.

We can observe from the plot of the previous task, that for the real data, all assortativity types are negative. This global property does describe the overall connectivity between the nodes. As we have seen in the lecture, social networks are usually assortative, whereas functional networks are usually negative assortatives i.e., disassortative. We can confirm this and this means, that means that similar vertices tend to *NOT* connect.

Taking the ERGM model into consideration, we can see, that the real mean for all assortativity types do not differ much (0.05 - 0.08). The plot *Comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks* suggests, that it is possible to model the assortativity of the original directed internet networks with the NEMtropy package, since we're inside the error bars. However, the undirected case is not true, which can be seen in the plot *Comparison of different assortativities of between a real graph and sampled ones over time for undirected internet networks*. This can also be implied from the *Timeline comparison of different assortativities of between a real graph and sampled ones over time for directed internet networks*, which shows that leveraging the ERGM model, the timeline of these internet graphs is a good approximation but only for the directed version.

Creating Results: `jupyter nbconvert Assignment_03.ipynb --to pdf --output Assignment_03_Results-T1.pdf`